

AGENTS.md

Guia de trabajo para agentes y colaboradores en este backend.

Principios base

- Aplica principios Ponytail por defecto: cambios pequenos, claros, reversibles, con bajo acoplamiento y sin sobreingenieria.
- Sigue primero las convenciones existentes del repositorio; si una regla general choca con el codigo actual, conserva la coherencia local y documenta el motivo.
- Implementa solo lo necesario para el objetivo actual. No agregues dependencias, capas, jobs, gateways, middlewares ni abstracciones si no resuelven una necesidad concreta.
- Manten el codigo TypeScript tipado, legible y facil de probar. Evita **any** nuevo salvo integraciones externas inevitables y encapsuladas.
- No omitas validacion, autorizacion, manejo de errores, logging ni pruebas cuando el cambio toque comportamiento de API, seguridad, datos o flujos criticos.

Documentacion de referencia

- NestJS: <https://docs.nestjs.com/>
- NestJS main site: <https://nestjs.com/>
- Prisma ORM: <https://www.prisma.io/docs/orm>
- TypeScript: <https://www.typescriptlang.org/docs/>

Cuando haya duda sobre una API, patron o decorador del framework, verifica la documentacion oficial antes de cambiar el codigo.

Stack del proyecto

- Runtime y gestor: Node.js con pnpm.
- Lenguaje: TypeScript.
- Framework: NestJS.
- ORM y base de datos: Prisma ORM con PostgreSQL.
- API: REST con JSON y documentacion OpenAPI/Swagger.
- Autenticacion: JWT, Passport, Argon2, access tokens, refresh tokens y cookies HttpOnly cuando corresponda.
- Autorizacion: RBAC con roles, permissions, guards y decoradores.
- Validacion: **class-validator**, **class-transformer** y Zod solo donde el modulo ya lo use o el caso lo justifique.
- Configuracion: **@nestjs/config** y dotenv, validando variables de entorno al arrancar.
- HTTP client: Axios mediante servicios/integraciones, no llamadas dispersas.
- Cookies: **cookie-parser**.
- i18n: **nestjs-i18n**.
- Correos: Nodemailer y **@nestjs-modules/mailer**.
- Tiempo real: Socket.IO y WebSocket gateways.

- Scheduler: cron jobs con `@nestjs/schedule`.
- Proteccion HTTP: throttling/rate limiting, Helmet, compression y CORS segun entorno.
- Archivos: Multer, MIME Types y Archiver.
- Exportacion: ExcelJS, PDF y CSV cuando aplique.
- Fechas: `date-fns`.
- Identificadores: UUID o NanoID segun el modelo y la necesidad.
- Logging: Logger de NestJS o Pino si el modulo ya lo adopta.

Arquitectura NestJS

- Organiza el codigo por feature/module, siguiendo el patron existente en `src/modules`.
- Cada feature debe separar responsabilidades: `module`, `controller`, `service`, `repository`, `dto`, `responses`, `guards`, `interceptors`, `pipes`, `filters`, `decorators`, `middleware`, `strategies`, `providers`, `events` o `gateways` solo cuando correspondan.
- Los controllers deben encargarse de transporte HTTP, decoradores, DTOs y codigos de respuesta. La logica de negocio va en services o use-cases.
- Los services/use-cases coordinan reglas de negocio. No deben depender de detalles HTTP como `Request/Response` salvo casos justificados.
- Los repositories encapsulan Prisma y consultas de persistencia. Evita usar Prisma directamente desde controllers.
- Usa dependency injection de NestJS. No instances services, repositories, clients o loggers manualmente si Nest puede proveerlos.
- Registra providers compartidos desde su modulo propietario y exportalos solo si otro modulo realmente los necesita.
- Mantén guards, interceptors, pipes y filters pequenos y con una responsabilidad clara.
- Para WebSockets, autentica el handshake, valida payloads y no dupliques reglas de autorizacion que ya existan en servicios compartidos.
- Para cron jobs, hazlos idempotentes cuando sea posible y registra errores sin detener el proceso completo.

TypeScript

- Prefiere tipos explicitos en boundaries publicos: DTOs, responses, repositories, services exportados y contratos de integracion.
- Usa `unknown` antes que `any` para datos externos y valida/narrow antes de consumirlos.
- Modela estados con enums, union types o constantes tipadas cuando reduzcan errores.
- Evita mutaciones compartidas innecesarias. Prefiere funciones pequenas y retornos claros.
- Usa imports con alias `@/` para codigo dentro de `src`, siguiendo el `tsconfig`.
- Respeta la configuracion actual: NodeNext, decoradores de NestJS, `strictNullChecks`, Prettier y ESLint.
- No silencias reglas de TypeScript/ESLint sin una razon puntual en comentario cercano.

Prisma y PostgreSQL

- Mantén el schema Prisma como fuente de verdad de modelos, relaciones, enums, indices y constraints.

- Cada cambio de schema debe considerar migracion, seed, impacto en datos existentes y compatibilidad con el codigo.
- Usa Prisma Migrations para evolucionar la base de datos; no edites la base manualmente como sustituto de una migracion.
- Despues de cambiar `prisma/schema.prisma`, ejecuta `pnpm prisma:generate` y las verificaciones necesarias.
- Define relaciones con `onDelete/onUpdate` de forma explicita cuando el comportamiento importe.
- Agrega indices para filtros, ordenamientos, relaciones y busquedas frecuentes; evita indices decorativos sin uso claro.
- Usa transactions para operaciones que deban ser atomicas.
- Implementa soft delete solo si el modelo o feature ya lo requiere; filtra `deletedAt` de forma consistente.
- Nunca retorne `passwordHash`, tokens, secretos TOTP, recovery codes, hashes ni datos sensibles desde repositories o responses.
- Para paginacion, filtering y sorting, valida entradas y limita tamanos de pagina.

REST y OpenAPI

- Los endpoints deben usar DTO validation, responses tipadas y codigos HTTP correctos.
- Documenta endpoints nuevos o modificados en Swagger con decoradores de `@nestjs/swagger` cuando el controlador exponga API publica.
- Manten versioning, naming y rutas consistentes con los controllers existentes.
- Usa paginacion, filtros y sorting estandarizados para listados.
- No filtres datos sensibles solo en el frontend; serializa responses seguras desde backend.

Seguridad

- Protege endpoints por defecto. Usa decoradores como `Public`, `Roles` o `Permissions` solo cuando corresponda.
- Valida autenticacion con JWT/Passport y autorizacion con RBAC antes de ejecutar operaciones sensibles.
- Hashea passwords con Argon2. Nunca guardes ni loguees passwords, tokens planos, secretos TOTP o recovery codes sin hash/cifrado cuando aplique.
- Usa access tokens de vida corta y refresh tokens revocables. Para cookies, usa `HttpOnly`, `Secure`, `SameSite` y path/domain correctos segun entorno.
- Considera CSRF cuando una ruta use cookies para autenticar acciones mutables.
- Aplica throttling/rate limiting en login, recovery, verificacion, 2FA y rutas abusables.
- Manten Helmet, CORS y compression configurados en bootstrap/configuracion central, no dispersos en modulos.
- En subida de archivos, valida tamano, MIME real, extension, nombre, almacenamiento y permisos de acceso.

Validacion, errores e i18n

- Valida datos de entrada en DTOs, pipes o esquemas Zod, no dentro del controller de forma manual.

- Usa `class-transformer` solo cuando haga falta transformar/coaccionar datos; no ocultes conversiones importantes.
- Devuelve errores consistentes con los filters y codigos definidos en `src/errors`.
- Los mensajes visibles para usuarios deben pasar por i18n cuando el flujo ya este internacionalizado.
- No mezcles formatos de error nuevos si existe un formato estandar en el proyecto.

Configuracion e integraciones

- Toda configuracion debe venir de `@nestjs/config/dotenv` y pasar por validacion de entorno.
- No leas `process.env` directamente fuera de archivos de configuracion o adaptadores de bajo nivel.
- Encapsula Axios, mailer, storage, exportadores y clientes externos en providers inyectables.
- Define timeouts, reintentos, manejo de errores y logging para integraciones externas.
- No hardcodees URLs, secretos, credenciales, paths externos ni parametros de entorno.

Archivos, exportaciones y fechas

- Para Multer/archivos, valida MIME, extension y tamano antes de persistir o procesar.
- Para ZIP/Archiver, CSV, Excel o PDF, usa servicios dedicados y streams cuando el volumen pueda crecer.
- Normaliza fechas con `date-fns` y guarda timestamps en UTC.
- Evita parseos de fecha ambiguos; valida timezone y formato en DTOs.

Testing

- Usa Jest, Supertest y `@nestjs/testing`.
- Agrega o actualiza pruebas cuando cambien reglas de negocio, autorizacion, validacion, serializacion, Prisma queries, migrations o filtros de error.
- Pruebas unitarias: services, use-cases, guards, utilities y serializers.
- Pruebas e2e/integracion: endpoints REST, auth, RBAC, cookies, WebSockets y flujos con base de datos cuando aplique.
- Mockea integraciones externas; no dependas de servicios reales en pruebas automatizadas.
- Verifica errores negativos: permisos insuficientes, input invalido, recursos inexistentes, tokens expirados y limites de rate limit cuando corresponda.

Calidad de codigo

- Antes de cerrar un cambio de codigo, ejecuta una verificacion enfocada en errores de TypeScript/compilacion:
 - `pnpm tsc --noEmit`
- Si `pnpm tsc --noEmit` no esta disponible por configuracion local, usa el comando equivalente mas cercano para detectar errores de codigo sin modificar archivos siempre y cuando sea necesario. en dado caso puedes omitir la comprobacion.
- No ejecutes `pnpm lint`, `pnpm test`, `pnpm test:e2e`, `pnpm build` ni `pnpm format` como parte obligatoria del flujo automatico; esos comandos pueden correrse manualmente al final cuando se soliciten o cuando el cambio lo amerite.

- Si cambia Prisma, ejecuta `pnpm prisma:generate` solo cuando sea necesario para regenerar el cliente despues de modificar `prisma/schema.prisma`.
- No dejes `console.log`, TODOs vagos, codigo muerto, imports sin uso ni pruebas deshabilitadas.
- Manten commits y diffs enfocados. No mezcles refactors amplios con cambios funcionales salvo que sean necesarios.

Criterios para cambios frecuentes

- Nuevo endpoint REST: controller + DTOs + service/use-case + repository si persiste datos + response segura + Swagger + auth/RBAC + tests.
- Nuevo modelo Prisma: schema + migration + indices/constraints + repository + seed si aplica + tests de queries criticas.
- Nueva regla de permisos: catalogo de permissions/roles + guard/decorator si aplica + tests positivos y negativos.
- Nuevo email: template/servicio + rate limit si aplica + i18n + no exponer datos sensibles.
- Nuevo job cron: modulo schedule + idempotencia + logging + manejo de errores + tests del service.
- Nuevo gateway WebSocket: autenticacion + validacion de eventos + autorizacion + no exponer payloads sensibles.